Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la ingeniería

Ingeniería

Organización de Lenguajes y Compiladores 1 Sección “A”

Ing. Jose Moises Granados Guevara

Primer Semestre 2026

Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

Manual técnico Proyecto #1

Descripción:

En el siguiente documento se da a conocer los detalles a tener en cuenta sobre el presente proyecto. Sobre todo si se desea ampliar las funcionalidades de la aplicación se deben de conocer a detalle cómo es que funciona el traductor y el intérprete realizado y sobre todo las clases y los diseños que fueron aplicados para poderla desarrollar.

“Bienvenido programador”

Como primer punto se requiere contar con los pasos para poder ejecutar el código del repositorio en la computadora local del programador, al igual que conocer la distribución de carpetas que se hizo.

Recomendaciones:

Debido a que la aplicación fue desarrollada en Android Studio se le recomienda al programador tener los siguientes requisitos:

- Android Studio o extensión de android en IntelliJIDEA
- Tener un teléfono Android o algun emulador instalado
- **Opcional:** Un teléfono con pantalla grande (esto con el motivo de tener una mejor visualización)

Pasos iniciales:

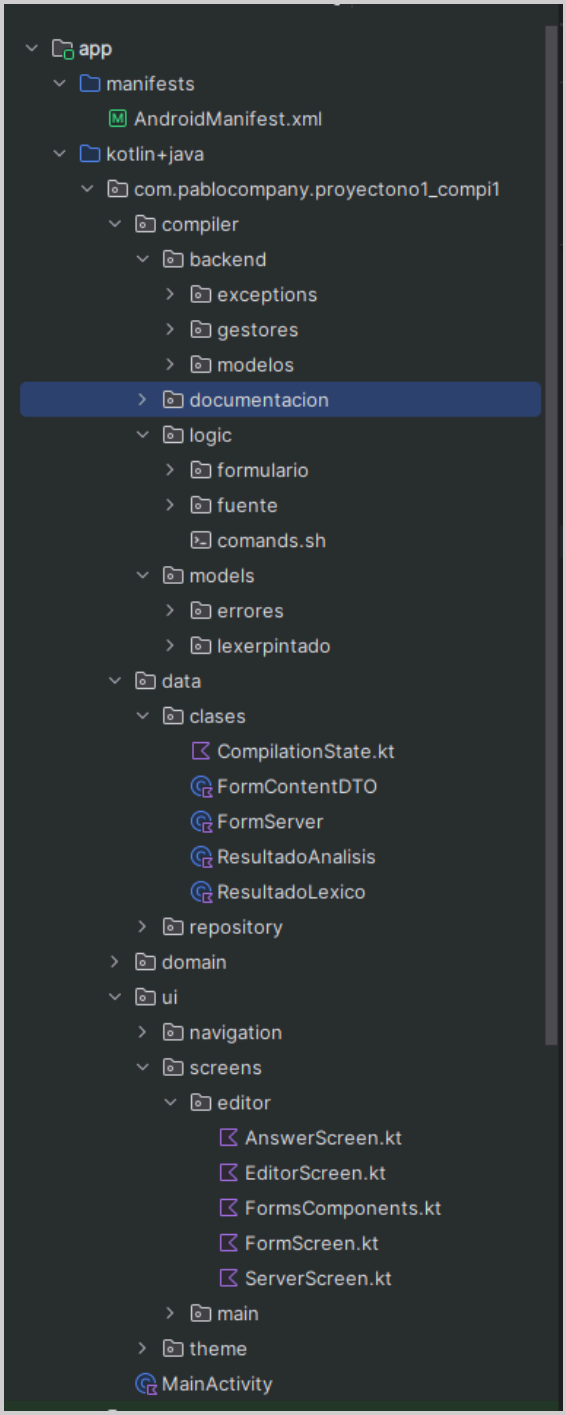
1. Encender la computadora.
2. Crear una carpeta donde se va a clonar el repositorio.
3. Ejecutar el comando:

   git clone https://github.com/pablo-mald03/Proyecto-No1-Compi-1.git
4. Tras descargar el repositorio se puede inicializar Android Studio.
5. Buscar la ruta de la carpeta donde fue clonado el repositorio y abrirla en Android Studio.
6. Al abrirlo se deben de aceptar los derechos

Si en dado caso se le pide descargar una extensión de **Cup.** Queda a discreción del programador si apoyarse de la extensión. Sin embargo no se recomienda ya que no tiene del todo las funciones necesarias y tiende a confundir.

DIRECTORIO DE ARCHIVOS

Debido a que el proyecto base se realizó con cierta distribución de archivos. Así que se le recomienda al nuevo programador que mantenga el estándar que se maneja en el nombramiento de variables.



Detalles importantes:

- **La carpeta "compiler":** es donde se encuentra toda la lógica del traductor e intérprete, de la demás lógica de negocio que maneja la aplicación.
- **La carpeta "data":** es donde se manejan los datos que se comparten vía enrutamiento a frontend par que este pueda interpretarlos
- **La carpeta: "usecase":** maneja los controladores que sirven de intermediarios para delegar al frontend instrucciones de qué hacer en base a las acciones.
- **La carpeta "screens":** maneja todas las vistas que se tienen pese a parecer pocas tienen muchas líneas de código cada una para poder armar las vistas generadas.

Paquete "ui"

Este paquete es el que gestiona por completo las vistas de la ui usando **JetpackCompose** por lo tanto se le piden al programador conocimientos básicos en esta opción de Android para que se puedan ampliar las vistas. **Importante siempre mantener el formato:**

MiVista**Screen.kt**

Este es un convenio general para evitar revolver código y confundir vistas sobre todo para futuros demás programadores.

IMPORTANTE:

Si se desean crear más paquetes es válido. Solo que si se debe de respetar la distribución base de los paquetes ya mencionados para seguir manteniendo el formato de orden. Sobre todo si hay que agregar documentación se le pide al programador que lo adjunte dentro del paquete llamado **documentación.**

ANÁLISIS DE GRAMÁTICA PARA ANALIZADOR LÉXICO

En el presente proyecto se lleva a cabo un análisis léxico gracias a JFlex que permite generar un **autómata finito determinista** en base a los principios del método de Thompson hasta poder llegar a un autómata con todos los estados que permita validar los estados de aceptación. Por lo tanto el método consiste en:

- Declaración de Expresiones regulares
- Generación de **AFN por el método de Thompson**
- Formulacion de **AFD en base al algoritmo de cerradura Lambda**
- **Minimizacion de estados**

En este caso este proyecto utilizó dos gramáticas distintas al igual que expresiones regulares diferentes ya que se requería uno para el lenguaje de programación y otro para el lenguaje de etiquetas que era interpretado por la otra gramática para poder trabajar en conjunto y generar las vistas respectivas. Por lo tanto a diferencia de ser un intérprete normal. Este realmente generaba código compilado o en este caso la palabra correcta es ser un traductor a otro lenguaje de etiquetas que permite generar los formularios que en este caso son archivos **.pkm**

ANALIZADOR LÉXICO CÓDIGO FUENTE

Aca se adjuntan todas las expresiones regulares utilizadas en el lexer:

Macros utilizados:

```
LineTerminator = \r\n|\r|\n

WhiteSpace = [ \t\f]+

Numero = [0-9]+

Decimal = {Numero}"."{Numero}

jletter = [:jletter:]

jletterdigit = [:jletterdigit:]

Id = {jletter}{jletterdigit}*

HexColor = "#"[0-9A-Fa-f]{6}



ComentarioBloque = "/*" ( [^*] | "*"+ [^/*] )* "*"+ "/"
```

Importante: En esta gramática se podrá observar que se tomaban en cuenta los espacios en blanco y comentarios debido a que se les tenía que dar color o como tal se tenían que procesar los tokens para poder generar un pintado dinámico del lenguaje de programación **.form**

Expresiones regulares utilizadas para el pintado dinamico:

```
/* Ignorar caracteres invisibles o de control no útiles */

[\u200B\u200C\u200D\uFEFF]        { /* ignorar */ }

/* Ignorar caracteres invisibles o de control no útiles */

[\p{C}&&[^\n\r\t]]      { /* ignorar */ }

/* Espacios y tabs */

{WhiteSpace}    {   if(!modoParser){
                       return symbol(sym.WHITESPACE, yytext());
                    }
                    /* ignorar */
                }

/* Saltos de línea */

{LineTerminator}    {   if(!modoParser){
                           return symbol(sym.NEWLINE, yytext());
                       }
                       /* ignorar */
                    }
```

```
/*--Comentarios--*/

"$".*      {    if(!modoParser){

                  return symbol(sym.COMENTARIO_TEXTO, yytext());

              }

              /* ignorar */

          }



{ComentarioBloque}      {   if(!modoParser){

                                        return  symbol(sym.COMENTARIO_TEXTO,
yytext());

                           }

                       }
```

Reconocimiento de colores:

```
/*---RECONOCIMIENTO DE COLORES---*/



{HexColor} { return symbol(sym.COLOR_HEX, yytext()); }

"RED"       { return symbol(sym.COLOR_PRESET, yytext()); }

"BLUE"      { return symbol(sym.COLOR_PRESET, yytext()); }

"GREEN"     { return symbol(sym.COLOR_PRESET, yytext()); }

"PURPLE"    { return symbol(sym.COLOR_PRESET, yytext()); }

"SKY"       { return symbol(sym.COLOR_PRESET, yytext()); }

"YELLOW"    { return symbol(sym.COLOR_PRESET, yytext()); }

"BLACK"     { return symbol(sym.COLOR_PRESET, yytext()); }

"WHITE"     { return symbol(sym.COLOR_PRESET, yytext()); }
```

Gramatica:

```
/*=========APARTADO DE ER QUE REPRESENTAN OPERADORES ARITMETICOS EN EL
LENGUAJE==========*/

"+" {return symbol(sym.SUMA, yytext());}

"-" {return symbol(sym.RESTA, yytext());}

"*" {return symbol(sym.MULTIPLICACION, yytext());}

"/" {return symbol(sym.DIVISION, yytext());}

"^" {return symbol(sym.POTENCIA, yytext());}

"%" {return symbol(sym.MODULO, yytext());}

"("       {return symbol(sym.PARENT_APERTURA, yytext());}

")"       {return symbol(sym.PARENT_CIERRE, yytext());}
```

```
/*=========APARTADO DE ER QUE REPRESENTAN OPERADORES DE COMPARACION EN EL
LENGUAJE==========*/

"=="    {return symbol(sym.IGUALDAD, yytext());}

"!!" {return symbol(sym.DIFERENTE, yytext());}

">"    {return symbol(sym.MAYOR, yytext());}

"<"    {return symbol(sym.MENOR, yytext());}

">="    {return symbol(sym.MAYOR_IGUAL, yytext());}

"<="    {return symbol(sym.MENOR_IGUAL, yytext());}

/*=========FIN DEL APARTADO DE ER QUE REPRESENTAN OPERADORES DE COMPARACION
EN EL LENGUAJE==========*/

/*=========APARTADO DE ER QUE REPRESENTAN OPERADORES DE LOGICOS EN EL
LENGUAJE==========*/

"&&"    {return symbol(sym.AND, yytext());}

"||"    {return symbol(sym.OR, yytext());}

"~"    {return symbol(sym.NOT, yytext());}
```

```
/*=========FIN DEL APARTADO DE ER QUE REPRESENTAN OPERADORES DE LOGICOS EN
EL LENGUAJE==========*/



/*=========APARTADO DE ER QUE REPRESENTAN LOS TIPOS DE VARIABLES EN EL
LENGUAJE==========*/

"number"        {return symbol(sym.VAR_NUMERO, yytext());}

"string"        {return symbol(sym.VAR_STRING, yytext());}

"special"       {return symbol(sym.VAR_ESPECIAL, yytext());}

/*=========APARTADO DE ER QUE REPRESENTAN LOS LAS DECLARACIONES DE VARIABLES
O TIENEN ALGUN CONTEXTO DE ASIGNACION EN EL LENGUAJE==========*/

"="    {return symbol(sym.IGUALACION, yytext());}

"?"    {return symbol(sym.COMODIN, yytext());}

","    {return symbol(sym.COMA, yytext());}

"["    {return symbol(sym.CORCHETE_APERTURA, yytext());}

"]"    {return symbol(sym.CORCHETE_CIERRE, yytext());}
```

```
"{"    {return symbol(sym.LLAVE_APERTURA, yytext());}

"}"    {return symbol(sym.LLAVE_CIERRE, yytext());}

"."    {return symbol(sym.PUNTO, yytext());}

":"    {return symbol(sym.DOS_PUNTOS, yytext());}

";"     {return symbol(sym.PUNTO_COMA, yytext());}

/*====***===APARTADO DE PALABRAS RESERVADAS DEL LENGUAJE====**====*/

"IF"    {return symbol(sym.IF, yytext());}

"ELSE"    {return symbol(sym.ELSE, yytext());}

"WHILE"    {return symbol(sym.WHILE, yytext());}

"FOR"    {return symbol(sym.FOR, yytext());}

"DO"    {return symbol(sym.DO, yytext());}
/*=====---****ATRIBUTO ESPECIAL DE LA SEGUNDA MANERA DEL FOR***---====*/
"in"    {return symbol(sym.IN, yytext());}
```

```
/*=========APARTADO EN EL QUE SE REGISTRAN LAS PALABRAS QUE CONFIGURAN
ESTILOS (COMPONENTES INICIALES)==========*/

"width"         {return symbol(sym.WIDTH, yytext());}

"height"         {return symbol(sym.HEIGHT, yytext());}

"label"         {return symbol(sym.LABEL, yytext());}

"pointX"         {return symbol(sym.POINT_X, yytext());}

"pointY"         {return symbol(sym.POINT_Y, yytext());}

"orientation"         {return symbol(sym.ORIENTATION, yytext());}

"elements"         {return symbol(sym.ELEMENTS, yytext());}

/*--Define estilos--*/
"styles"    {return symbol(sym.STYLES, yytext());}

/*Define el contenido de un texto*/
"content"    {return symbol(sym.CONTENT, yytext());}
```

```
/*------EXPRESIONES CON UN CONTEXTO ESPECIAL PARA LOS ESTILOS DEL CODIGO
COMPILADO-------*/

"\"color\""             {return symbol(sym.COLOR_TEXTO, yytext());}

"\"background  color\""                {return  symbol(sym.BACKGROUND_COLOR,
yytext());}

"\"font family\""              {return symbol(sym.FONT_FAMILY, yytext());}

"\"text size\""         {return symbol(sym.TEXT_SIZE, yytext());}

"\"border\""         {return symbol(sym.BORDER, yytext());}


/*------EXPRESIONES CON UN CONTEXTO ESPECIAL PARA GENERACION DEL CODIGO
COMPILADO-------*/

"draw"      {return symbol(sym.DRAW, yytext());}

"who_is_that_pokemon" {return symbol(sym.WHO_IS_THAT_POKEMON, yytext());}

"NUMBER"        {return symbol(sym.NUMBER_REQUEST, yytext());}

/*---CONFIGURACIONES DE DOCK---*/
"VERTICAL"      {return symbol(sym.CONFIG_DOCK, yytext());}
```

```
"HORIZONTAL"    {return symbol(sym.CONFIG_DOCK, yytext());}

/*Created by Pablo*/
/*---TIPOGRAFIAS DE LETRAS---*/

"MONO"      {return symbol(sym.TIPOGRAFIA, yytext());}

"SANS_SERIF"      {return symbol(sym.TIPOGRAFIA, yytext());}

"CURSIVE"     {return symbol(sym.TIPOGRAFIA, yytext());}

/*---GROSOR DE LINEA---*/
"LINE"      {return symbol(sym.GROSOR_LINEA, yytext());}

"DOTTED"    {return symbol(sym.GROSOR_LINEA, yytext());}

"DOUBLE"    {return symbol(sym.GROSOR_LINEA, yytext());}


/*=========APARTADO  EN  EL  QUE  SE  REGISTRAN  LAS  PALABRAS  RESERVADAS
(PREGUNTAS O CONFIGURACIONES DEL FORMULARIO)==========*/

"SECTION"               {return symbol(sym.SECTION, yytext());}

"TABLE"                 {return symbol(sym.TABLE, yytext());}
```

```
"TEXT"                  {return symbol(sym.TEXT, yytext());}



/*--*--Seccion de perguntas--*--*/

"OPEN_QUESTION"         {return symbol(sym.OPEN_QUESTION, yytext());}


"DROP_QUESTION"         {return symbol(sym.DROP_QUESTION, yytext());}


"SELECT_QUESTION"       {return symbol(sym.SELECT_QUESTION, yytext());}


"MULTIPLE_QUESTION"     {return symbol(sym.MULTIPLE_QUESTION, yytext());}

/*------*---Apartado de contenido que tienen las preguntas----*--------*/

"correct"               {return symbol(sym.CORRECT, yytext());}


"options"               {return symbol(sym.OPTIONS, yytext());}


{Decimal}  {return symbol(sym.DECIMAL, Double.parseDouble(yytext()));}


{Numero} {return symbol(sym.ENTERO, Integer.parseInt(yytext()));}


{Id} { return symbol(sym.ID, yytext()); }
```

Reconocimiento de cadenas de texto con emojis:

```
/*-----FIN DE LA CADENA DE TEXTO-----*/

\" {
   yybegin(YYINITIAL);
   return symbol(sym.FIN_CADENA, yytext());
}

/*-----EMOJIS DINAMICOS-----*/
( "@[:" ")"+ "]" | "@[:smile:]" )          { return symbol(sym.EMOJI_SMILE,
yytext()); }

( "@[:" "("+ "]" | "@[:sad:]" )             { return symbol(sym.EMOJI_SAD,
yytext()); }

( "@[:" "|"+ "]" | "@[:serious:]" )       { return symbol(sym.EMOJI_SERIOUS,
yytext()); }

( "@[" "<"+ "3"+ "]" | "@[:heart:]" )     { return symbol(sym.EMOJI_HEART,
yytext()); }

"@[:star:]"                                  { return symbol(sym.EMOJI_STAR,
yytext()); }

( "@[:star:" {Numero} ":]" | "@[:star-" {Numero} ":]" )        { return
symbol(sym.EMOJI_MULTI_STAR, yytext()); }
```

```
( "@[:^^:]" | "@[:cat:]" )                    { return symbol(sym.EMOJI_CAT,
yytext()); }



/*-----Texto normal (OBVIA @):-----*/


[^\n\r\"\\@]+ { return symbol(sym.TEXTO_PLANO, yytext()); }


/*-----RECONOCE COMO TEXTO NORMAL A @-----*/


"@" { return symbol(sym.TEXTO_PLANO, "@"); }


/*-----Escapes de cadenas siguen devolviendo texto plano------*/


\\\" { return symbol(sym.TEXTO_PLANO, "\""); }
\\n  { return symbol(sym.TEXTO_PLANO, "\n"); }
```

Estados utilizados:

```
<YYINITIAL>{

}

<STRING> {


 }



<<EOF>>        {

                return symbol(sym.EOF);

            }

/*Created by Pablo*/
```

ANALIZADOR LÉXICO CÓDIGO FUENTE

Aca se adjuntan todas las expresiones regulares utilizadas en el lexer:

Macros utilizados:

```
LineTerminator = \r\n|\r|\n

WhiteSpace = [ \t\f]+

Numero = [0-9]+

Decimal = {Numero}"."{Numero}



//Metadatos



MetaDatos = "###" ( [^#] | "#" [^#] | "##" [^#] )* "###"



HexColor = "#"[0-9A-Fa-f]{6}
```

Importante: Es importante mencionar que en este caso ya no se reconocían tokens de espacios simplemente se ignoraban al igual que los metadatos.

Reconocimiento de colores:

```
/*---APARTADO DE COLORES---*/


{HexColor} { return symbol(SymCompiled.COLOR_HEX, yytext()); }


/*---RECONOCIMIENTO DE COLORES---*/


"RED"       { return symbol(SymCompiled.COLOR_PRESET, yytext()); }

"BLUE"      { return symbol(SymCompiled.COLOR_PRESET, yytext()); }

"GREEN"     { return symbol(SymCompiled.COLOR_PRESET, yytext()); }

"PURPLE"    { return symbol(SymCompiled.COLOR_PRESET, yytext()); }

"SKY"       { return symbol(SymCompiled.COLOR_PRESET, yytext()); }

"YELLOW"    { return symbol(SymCompiled.COLOR_PRESET, yytext()); }

"BLACK"     { return symbol(SymCompiled.COLOR_PRESET, yytext()); }

"WHITE"     { return symbol(SymCompiled.COLOR_PRESET, yytext()); }
```

Gramatica:

```
/* Ignorar caracteres invisibles o de control no útiles */

[\u200B\u200C\u200D\uFEFF]        { /* ignorar */ }

/* Ignorar caracteres invisibles o de control no útiles */

[\p{C}&&[^\n\r\t]]  { /* ignorar */ }

/* Espacios y tabs */

{WhiteSpace} { /*ignorar*/ }

/* Saltos de línea */

{LineTerminator} { /*ignorar*/ }

/*Ignorar metadatos*/

{MetaDatos}     {/*ignorar*/}


/*=====***--DEFINICION DE EXPRESIONES MATEMATICAS-***====*/

"-" {return symbol(SymCompiled.RESTA, yytext());}
```

```
/*=====***--DEFINICION DE EXPRESIONES MATEMATICAS-***====*/



/*=====***--DEFINICION DE CARACTERES ESPECIALES DE ETIQUETAS--***====*/

"="      { return symbol(SymCompiled.IGUAL); }

","      { return symbol(SymCompiled.COMA); }

"{"      { return symbol(SymCompiled.LLAVE_APERTURA); }

"}"      { return symbol(SymCompiled.LLAVE_CIERRE); }

/*=====***--DEFINICION DE CARACTERES ESPECIALES DE ETIQUETAS--***====*/

/*---APARTADO DE PALABRAS RESERVADAS---*/

"style"      { return symbol(SymCompiled.STYLE); }

"content"    { return symbol(SymCompiled.CONTENT); }

"color"      { return symbol(SymCompiled.COLOR_KEY); }

"background" { return symbol(SymCompiled.BACKGROUND); }
```

```
"font"       { return symbol(SymCompiled.FONT); }

"family"     { return symbol(SymCompiled.FAMILY); }

"size"       { return symbol(SymCompiled.SIZE); }

"border"     { return symbol(SymCompiled.BORDER); }

"section"    { return symbol(SymCompiled.SECTION); }

"table"      { return symbol(SymCompiled.TABLE); }

"text"       { return symbol(SymCompiled.TEXT); }

"line"       { return symbol(SymCompiled.LINE, yytext()); }

"element"    { return symbol(SymCompiled.ELEMENT); }

"open"       { return symbol(SymCompiled.OPEN); }

"drop"       { return symbol(SymCompiled.DROP); }

"select"     { return symbol(SymCompiled.SELECT); }

"multiple"   { return symbol(SymCompiled.MULTIPLE); }
```

```
/*=====***--FIN DEL APARTADO DE ETIQUETAS COMPUESTAS--***====*/



/*------****--------APARTADO DE SIMBOLOS ESPECIALES--------****------*/

"/>"     { return symbol(SymCompiled.CIERRE_ETIQUETA); }

"</"     { return symbol(SymCompiled.INICIO_ETIQUETA); }

"("       {return symbol(SymCompiled.PARENT_APERTURA);}

")"       {return symbol(SymCompiled.PARENT_CIERRE);}

"<"      { return symbol(SymCompiled.MENOR); }

">"      { return symbol(SymCompiled.MAYOR); }


/*------****--------FIN        DEL        APARTADO        DE        SIMBOLOS
ESPECIALES--------****------*/
```

```
/*=====***--DEFINICION DE TIPOS DE LETRA Y PRESETS--***====*/


/*---CONFIGURACIONES DE DOCK---*/

"VERTICAL"      {return symbol(SymCompiled.CONFIG_DOCK, yytext());}


"HORIZONTAL"    {return symbol(SymCompiled.CONFIG_DOCK, yytext());}


/*---TIPOGRAFIAS DE LETRAS---*/


"MONO"      {return symbol(SymCompiled.TIPOGRAFIA, yytext());}


"SANS_SERIF"      {return symbol(SymCompiled.TIPOGRAFIA, yytext());}


"CURSIVE"     {return symbol(SymCompiled.TIPOGRAFIA, yytext());}



/*---GROSOR DE LINEA---*/


"DOTTED"    {return symbol(SymCompiled.GROSOR_LINEA, yytext());}


"DOUBLE"    {return symbol(SymCompiled.GROSOR_LINEA, yytext());}



/*=====***--FIN DE LA DEFINICION DE TIPOS DE LETRA Y PRESETS--***====*/
```

```
/*==========ER CON CONTEXTO DE VALORES EN EL LENGUAJE============*/

{Decimal}                        {return            symbol(SymCompiled.DECIMAL,
Double.parseDouble(yytext()));}

{Numero} {return symbol(SymCompiled.ENTERO, Integer.parseInt(yytext()));}

/*----RECONOCIMIENTO DE CADENAS DE TEXTO-----*/

\"          { yybegin(STRING);
             return symbol(SymCompiled.INICIO_CADENA, yytext()); }
```

Reconocimiento de cadenas de texto con emojis:

```
/*-----FIN DE LA CADENA DE TEXTO-----*/

\" {
   yybegin(YYINITIAL);
   return symbol(sym.FIN_CADENA, yytext());
}

/*-----EMOJIS DINAMICOS-----*/
(  "@[:"  ")"+  "]"  |  "@[:smile:]"  )          {  return  symbol(sym.EMOJI_SMILE,
yytext()); }
```

```
( "@[:" "("+ "]" | "@[:sad:]" )             { return symbol(sym.EMOJI_SAD,
yytext()); }


( "@[:" "|"+ "]" | "@[:serious:]" )       { return symbol(sym.EMOJI_SERIOUS,
yytext()); }


( "@[" "<"+ "3"+ "]" | "@[:heart:]" )     { return symbol(sym.EMOJI_HEART,
yytext()); }


"@[:star:]"                                  { return symbol(sym.EMOJI_STAR,
yytext()); }


( "@[:star:" {Numero} ":]" | "@[:star-" {Numero} ":]" )        { return
symbol(sym.EMOJI_MULTI_STAR, yytext()); }


( "@[:^^:]" | "@[:cat:]" )                    { return symbol(sym.EMOJI_CAT,
yytext()); }



/*-----Texto normal (OBVIA @):-----*/


[^\n\r\"\\@]+ { return symbol(sym.TEXTO_PLANO, yytext()); }


/*-----RECONOCE COMO TEXTO NORMAL A @-----*/


"@" { return symbol(sym.TEXTO_PLANO, "@"); }
```

```
/*-----Escapes de cadenas siguen devolviendo texto plano------*/



\\\" { return symbol(sym.TEXTO_PLANO, "\""); }

\\n  { return symbol(sym.TEXTO_PLANO, "\n"); }
```

Estados utilizados:

<YYINITIAL>{

}

<STRING> {

}

<<EOF>> {

return symbol(sym.EOF);

}

/*Created by Pablo*/

ANÁLISIS DE GRAMÁTICA PARA ANALIZADOR SINTÁCTICO PRINCIPAL

Cabe aclarar que aquí simplemente se están agregando los **terminales** y los **no terminales** debido a que la gramática es muy extensa. pero simplemente se puede visualizar la cantidad de derivaciones que esta tiene. Puesto a que los errores se debían detectar desde el Parser el código es bien extenso. Por lo tanto para más información consultar el archivo llamado.

ParserFormConfig.cup

```
/* Terminales: tokens que el parser nos retorna */

terminal    ERROR, WHITESPACE, NEWLINE,COMENTARIO_TEXTO,

           UNARIO,SUMA, RESTA, MULTIPLICACION, DIVISION, POTENCIA, MODULO,

           PARENT_APERTURA, PARENT_CIERRE,

           IGUALDAD, DIFERENTE, MAYOR, MENOR, MAYOR_IGUAL, MENOR_IGUAL,

           AND, OR, NOT,

           VAR_NUMERO, VAR_STRING, VAR_ESPECIAL,

           IGUALACION, COMODIN, COMA,

           INICIO_CADENA, FIN_CADENA,

           CORCHETE_APERTURA, CORCHETE_CIERRE,

           LLAVE_APERTURA, LLAVE_CIERRE,

           IF, ELSE, WHILE, FOR, DO, IN,

           PUNTO, DOS_PUNTOS, PUNTO_COMA,

           WIDTH, HEIGHT, LABEL, POINT_X, POINT_Y,ORIENTATION,

           ELEMENTS,

           STYLES, CONTENT,

           BACKGROUND_COLOR, COLOR_TEXTO, FONT_FAMILY, TEXT_SIZE,BORDER,
```

```
            DRAW, WHO_IS_THAT_POKEMON,

            SECTION, TABLE, TEXT,

            OPEN_QUESTION, DROP_QUESTION, SELECT_QUESTION, MULTIPLE_QUESTION,

            CORRECT, OPTIONS, NUMBER_REQUEST

            ;


/*Created by Pablo*/

/*======Apartado de tipado de tokens que vienen del lexer======*/

terminal String ID,COLOR_HEX,TEXTO_PLANO;

terminal String TIPOGRAFIA,GROSOR_LINEA, CONFIG_DOCK,COLOR_PRESET;

terminal   String   EMOJI_SMILE,   EMOJI_SAD,   EMOJI_SERIOUS,   EMOJI_HEART,
EMOJI_STAR, EMOJI_MULTI_STAR, EMOJI_CAT;

terminal Double DECIMAL;

terminal Integer ENTERO;


terminal Integer ENTERO;

terminal String CADENA;
```

Cabe destacar algo muy importante:

Al usar expresiones matemáticas esto causaba Shift-Reduce. Es decir que la gramática se hacía ambigua al no indicarle a cup cuál era la precedencia de operadores. Por lo tanto se optó por la herramienta de **precedencia.** Que permite indicar la dirección de la precedencia que siguen los operadores.

```
precedence left OR;

precedence left AND;

precedence left SUMA, RESTA;

precedence left MULTIPLICACION, DIVISION;

precedence left MODULO;

precedence right POTENCIA;

precedence right NOT;

precedence right UNARIO;
```

IMPORTANTE:

Debido a que la precedencia de los operadores lógicos también se requiere saber por donde genera su efecto y debido a que el operador NOT casi siempre se aplica en una expresión este es el que debe de tener la principal precedencia en la jerarquia. Por lo tanto la jerarquia queda asi:

~ (NOT) se evalúa primero

&& (AND) después

|| (OR) al final

Producciones más importantes:

```
formulario          ::= lista_codigo:list

                    ;



lista_codigo        ::= lista_codigo:list unidad_codigo:uni

                    | unidad_codigo:uni

                        ;





unidad_codigo       ::= instruccion:ins

                    | section_declaration:sec

                    | table_declaration:tab

                    | condicional:con

                    | ciclo_for:forCicloleft

                    | ciclo_while:whileCiclo

                    | ciclo_do_while:doCiclo

                    | error

                    ;



ciclo_for               ::=  FOR:forCiclo  PARENT_APERTURA  ID:id1  IGUALACION
expresion:exp1   PUNTO_COMA   condicion:cond   PUNTO_COMA   ID:id2   IGUALACION
expresion:exp2

                                                PARENT_CIERRE  LLAVE_APERTURA
lista_codigo_bloque:cuerpo LLAVE_CIERRE

                      | FOR:forCiclo PARENT_APERTURA ID:id IN expresion:exp1
PUNTO      PUNTO      expresion:exp2      PARENT_CIERRE      LLAVE_APERTURA
lista_codigo_bloque:cuerpo LLAVE_CIERRE
```

```
                    ;



/*-----REGLAS DE PRODUCCION PARA DECLARACION DEL CICLO WHILE-----*/


ciclo_while              ::= WHILE:whileCiclo PARENT_APERTURA condicion:cond
PARENT_CIERRE LLAVE_APERTURA lista_codigo_bloque:cuerpo LLAVE_CIERRE

                        ;


/*-----REGLAS DE PRODUCCION PARA DECLARACION DEL CICLO DO WHILE-----*/


ciclo_do_while            ::= DO LLAVE_APERTURA lista_codigo_bloque:cuerpo
LLAVE_CIERRE WHILE:whileCiclo PARENT_APERTURA condicion:cond PARENT_CIERRE

                        ;





/*-----REGLAS DE PRODUCCION PARA DECLARACION DE CONDICIONALES EN EL CODIGO
FUENTE-----*/

condicional              ::= IF PARENT_APERTURA condicion:con PARENT_CIERRE
LLAVE_APERTURA lista_codigo_bloque:cuerpo LLAVE_CIERRE

                          | IF PARENT_APERTURA condicion:con PARENT_CIERRE
LLAVE_APERTURA lista_codigo_bloque:cuerpo LLAVE_CIERRE bloque_else:el


                    ;


/*-----REGLAS DE PRODUCCION PARA DECLARACION DE CONDICIONALES EN EL CODIGO
FUENTE-----*/
```

```
bloque_else             ::= ELSE LLAVE_APERTURA lista_codigo_bloque:cuerpo
LLAVE_CIERRE
                   | ELSE condicional:ifAnidado
                   ;




/*-----REGLAS DE PRODUCCION PARA LAS ACCIONES QUE PUEDEN HACER LAS
CODICIONES Y CICLOS DENTRO DE ELLOS-----*/


/*Define la derivacion principal para el reocnocimiento de codigo fuente
dentro de ciclos y condicionales*/


lista_codigo_bloque   ::= lista_codigo_bloque :list unidad_codigo_bloque:uni
                          | unidad_codigo_bloque:uni
                          ;



/*-----Define la derivacion principal de intrucciones para el reocnocimiento
de codigo fuente dentro de ciclos y condicionales-----*/


unidad_codigo_bloque       ::= instruccion_bloque:ins
                           | section_declaration:sec
                           | table_declaration:tab
                           | condicional:con
                           | ciclo_for:forCicloleft
```

```
                          | ciclo_while:whileCiclo

                          | ciclo_do_while:doCiclo

                          | error

                          ;


/*-----Unica instruccion que se puede hacer dentro de un ciclo----*/


instruccion_bloque        ::= ID:id IGUALACION expresion:expresion

                          ;




/*Created by Pablo*/

/*-----REGLAS DE PRODUCCION PARA DECLARACION DE CONDICIONES-----*/


condicion       ::= condicion:condA AND:and condicion:condB

                  | condicion:condA OR:or condicion:condB

                  | NOT:not condicion:condA

                  | PARENT_APERTURA condicion:condA PARENT_CIERRE

                  | expresion:expA IGUALDAD expresion:expB

                  | expresion:expA DIFERENTE expresion:expB

                  | expresion:expA MAYOR expresion:expB

                  | expresion:expA MENOR expresion:expB

                  | expresion:expA MAYOR_IGUAL expresion:expB

                  | expresion:expA MENOR_IGUAL expresion:expB

                  ;
```

```
estilo_individual       ::= BACKGROUND_COLOR DOS_PUNTOS color_valor:color

                        | COLOR_TEXTO DOS_PUNTOS color_valor:color

                        | FONT_FAMILY DOS_PUNTOS TIPOGRAFIA:tipo

                        | TEXT_SIZE DOS_PUNTOS expresion:exp


                        ;


/*====Reglas que permiten definir los colores===*/

color_valor     ::= COLOR_HEX:hex

                              |  PARENT_APERTURA:parent  expresion:exp1  COMA
expresion:exp2 COMA expresion:exp3 PARENT_CIERRE

                    | MENOR:apertura expresion:exp1 COMA expresion:exp2 COMA
expresion:exp3 MAYOR

                  | COLOR_PRESET:preset

                  ;




/*Created by Pablo*/

/*----Expresiones del lenguaje-----*/

expresion ::=       expresion:exp1 SUMA expresion:exp2

                  | expresion:exp1 RESTA expresion:exp2

                  | RESTA expresion:exp1

                  %prec UNARIO

                  | expresion:exp1 MULTIPLICACION expresion:exp2

                  | expresion:exp1 DIVISION expresion:exp2
```

```
                    | expresion:exp1 MODULO expresion:exp2

                    | expresion:exp1 POTENCIA expresion:exp2

                    | PARENT_APERTURA expresion:exp PARENT_CIERRE

                    | COMODIN:comodin

                    | ENTERO:entero

                    | DECIMAL:dec

                    | ID:id

                    | INICIO_CADENA lista_fragmentos:list FIN_CADENA

                    | INICIO_CADENA:inCad FIN_CADENA

                    ;


//----*----Representa las cadenas con fragmentos de texto y emojis----*----


lista_fragmentos    ::= lista_fragmentos:list fragmento:frag

                    | fragmento:frag

                    | lista_fragmentos:list error

                    ;


//----*----Representa cada uno de los fragmentos de texto y emojis que
vienen dentro de una cadena de texto----*----

fragmento           ::= TEXTO_PLANO:texto

                    | EMOJI_SMILE:emoji

                    | EMOJI_SAD:emoji

                    | EMOJI_SERIOUS:emoji

                    | EMOJI_HEART:emoji

                    | EMOJI_STAR:emoji
```

```
                    | EMOJI_MULTI_STAR:emoji

                    | EMOJI_CAT:emoji

                    ;




/*Created by Pablo*/
```

ANÁLISIS DE GRAMÁTICA PARA ANALIZADOR SINTÁCTICO PRINCIPAL

Cabe aclarar que aquí simplemente se están agregando los **terminales** y los **no terminales** debido a que la gramática es muy extensa. pero simplemente se puede visualizar la cantidad de derivaciones que esta tiene. Puesto a que los errores se debían detectar desde el Parser el código es bien extenso. Por lo tanto para más información consultar el archivo llamado.

ParserCompiledConfig.cup

Es importante también destacar que este parser es mucho más simple ya que no requiere de ciertas operaciones ya que el código compilado ya solo es interpretado para poder generar los formularios.

Tokens:

```
/* Terminales: tokens que el parser nos retorna */

terminal    RESTA,

           IGUAL, COMA, LLAVE_APERTURA, LLAVE_CIERRE,

               STYLE, CONTENT, COLOR_KEY, BACKGROUND, FONT, FAMILY, SIZE,
BORDER,

           SECTION, TABLE, TEXT, ELEMENT, OPEN, DROP, SELECT, MULTIPLE,

           CIERRE_ETIQUETA, INICIO_ETIQUETA, MENOR, MAYOR,

           PARENT_APERTURA, PARENT_CIERRE,

           INICIO_CADENA, FIN_CADENA

           ;
```

```
/*======Apartado de tipado de tokens que vienen del lexer======*/

terminal String COLOR_HEX;

terminal Double DECIMAL;

terminal Integer ENTERO;

terminal String GROSOR_LINEA, LINE, TIPOGRAFIA;

terminal String COLOR_PRESET;

terminal String CONFIG_DOCK;
```

Gramatica:

```
/* Simbolo inicial */

start with compilado;





/*Created by Pablo*/




compilado           ::= cuerpo_etiquetas:list

                    ;


/*Produccon del cuerpo de etiquetas*/

cuerpo_etiquetas        ::=  cuerpo_etiquetas:list etiqueta_principal:e
```

```
                        | etiqueta_principal:e

                        | error

                        | cuerpo_etiquetas:list error

                        ;




/*Produccion que representa al inicio del codigo interpretado*/


etiqueta_principal      ::=  etiqueta_tabla:tabla

                        | etiqueta_section:sec

                        ;




/*Produccion que permite interpretar una section*/


etiqueta_section          ::= MENOR SECTION:sec IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_numerico:pointx COMA valor_numerico:pointy COMA
CONFIG_DOCK:orient MAYOR

                                bloque_estilos:estilos MENOR CONTENT MAYOR
etiquetas_sections:et  INICIO_ETIQUETA CONTENT MAYOR INICIO_ETIQUETA SECTION
MAYOR

                            | MENOR SECTION:sec IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_numerico:pointx COMA valor_numerico:pointy COMA
CONFIG_DOCK:orient MAYOR

                                 MENOR CONTENT MAYOR etiquetas_sections:et
INICIO_ETIQUETA CONTENT MAYOR INICIO_ETIQUETA SECTION MAYOR
```

```
                        ;



/*--Produccion que permite interpretar a los valores que puede tener una
section--*/


etiquetas_sections        ::=  etiquetas_sections:list etiqueta:e


                           | etiqueta:e

                           | etiquetas_sections:list error

                           ;



/*Produccion que permite interpretar una tabla*/


etiqueta_tabla      ::=   MENOR   TABLE:tab   IGUAL   valor_numerico:w   COMA
valor_numerico:h COMA valor_numerico:pointx COMA valor_numerico:pointy MAYOR
bloque_estilos:estilos     MENOR     CONTENT     MAYOR     filas_tabla:filas
INICIO_ETIQUETA CONTENT MAYOR INICIO_ETIQUETA TABLE MAYOR

                             | MENOR  TABLE:tab  IGUAL  valor_numerico:w  COMA
valor_numerico:h COMA valor_numerico:pointx COMA valor_numerico:pointy MAYOR
MENOR  CONTENT  MAYOR  filas_tabla:filas    INICIO_ETIQUETA  CONTENT  MAYOR
INICIO_ETIQUETA TABLE MAYOR

                    ;
```

```
/*Produccion que permite evaluar a todas las filas que vengan dentro de la
tabla*/


filas_tabla     ::=  filas_tabla:list fila:f

                   | fila:f

                   | filas_tabla:list error

                   ;





/*Produccion que permite crear una fila*/


fila                  ::= MENOR LINE MAYOR celdas_tabla:fil INICIO_ETIQUETA
LINE MAYOR

                   ;





/*Produccion que permite reconocer las columnas de la tabla*/


celdas_tabla       ::=  celdas_tabla:list celda:c

                   | celda:c

                   ;


/*Produccion que permite reconocer a las celdas*/
```

```
celda                    ::=  MENOR ELEMENT MAYOR etiqueta:e INICIO_ETIQUETA
ELEMENT MAYOR

                    | MENOR ELEMENT error MAYOR

                    ;





/*Produccion que representa a todos los elementos que pueden tener dentro
unas secciones o tablas*/


etiqueta                 ::=  etiqueta_tabla:tabla


                       | etiqueta_section:sec

                       | etiqueta_text:texto

                       | etiqueta_open:openQ

                       | etiqueta_drop: dropQ

                       | etiqueta_select:selQ

                       | etiqueta_multiple:multQ

                       ;





/*-*----*-----*-----Producciones para los textos----*------*-----*-*/


etiqueta_text             ::=  MENOR  TEXT:txt  IGUAL  valor_numerico:w  COMA
valor_numerico:h COMA valor_cadena:cad CIERRE_ETIQUETA
```

```
                                | MENOR TEXT:txt IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad MAYOR

                        bloque_estilos_quests:est INICIO_ETIQUETA TEXT MAYOR

                    ;









/*-*----*-----*-----Producciones         para         las         preguntas
open----*------*-----*-*/


etiqueta_open              ::= MENOR OPEN:op IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad CIERRE_ETIQUETA

                                | MENOR OPEN:op IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad MAYOR

                        bloque_estilos_quests:est INICIO_ETIQUETA OPEN MAYOR

                    ;




/*-*----*-----*-----Producciones         para         las         preguntas
drop----*------*-----*-*/


etiqueta_drop                ::= MENOR DROP:dr IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad COMA

                                            LLAVE_APERTURA lista_opciones:ops
LLAVE_CIERRE COMA valor_numerico:def CIERRE_ETIQUETA

                                | MENOR DROP:dr IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad COMA
```

```
                                LLAVE_APERTURA lista_opciones:ops LLAVE_CIERRE
COMA valor_numerico:def MAYOR

                                bloque_estilos_quests:est INICIO_ETIQUETA DROP
MAYOR

                        ;




/*-*----*-----*-----Producciones         para         las         preguntas
select----*------*-----*-*/


etiqueta_select            ::= MENOR SELECT:sele IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad COMA

                                             LLAVE_APERTURA lista_opciones:ops
LLAVE_CIERRE COMA valor_numerico:def CIERRE_ETIQUETA

                              | MENOR SELECT:sele IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad COMA

                                             LLAVE_APERTURA lista_opciones:ops
LLAVE_CIERRE COMA valor_numerico:def MAYOR

                                   bloque_estilos_quests:est INICIO_ETIQUETA
SELECT MAYOR

                           ;




/*-*----*-----*-----Producciones         para         las         preguntas
multiple----*------*-----*-*/


etiqueta_multiple        ::= MENOR MULTIPLE:mult IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad COMA
```

```
                                LLAVE_APERTURA lista_opciones:ops LLAVE_CIERRE
COMA

                            lista_respuestas:res CIERRE_ETIQUETA

                           | MENOR MULTIPLE:mult IGUAL valor_numerico:w COMA
valor_numerico:h COMA valor_cadena:cad COMA

                          LLAVE_APERTURA lista_opciones:ops LLAVE_CIERRE COMA
lista_respuestas:res MAYOR

                           bloque_estilos_quests:est INICIO_ETIQUETA MULTIPLE
MAYOR

                        ;





/*----Lista de opciones que puede tener la drop question-----*/


lista_opciones      ::= lista_opciones:list COMA valor_cadena:cad

                    | valor_cadena:cad

                    ;


/*----Lista de respuestas que puede tener la multiple question-----*/


lista_respuestas                ::=  LLAVE_APERTURA  valores_respuestas:list
LLAVE_CIERRE

                        | LLAVE_APERTURA LLAVE_CIERRE

                        ;
```

```
/*----Lista de las diferentes respuestas que puede tener la multiple
question-----*/


valores_respuestas      ::= valores_respuestas:list COMA valor_numerico:num

                        | valor_numerico:num


                        ;




/*-*----*-----*-----Producciones para las preguntas----*------*-----*-*/




/*-----Producciones para estilos de layouts (tablas o sections)-----*/


/*---Produccion principal----*/


bloque_estilos              ::=  MENOR  STYLE  MAYOR  lista_estilos:estilos
INICIO_ETIQUETA STYLE MAYOR

                   | error MAYOR

                   ;


bloque_estilos_quests                        ::=    MENOR    STYLE    MAYOR
lista_estilos_quests:estilos INICIO_ETIQUETA STYLE MAYOR

                           | error MAYOR

                           ;
```

```
/*---Produccion que permite definir los estilos----*/


lista_estilos       ::= lista_estilos:list lista_propiedades:est

                    | lista_propiedades:est


                    | lista_estilos:est error


                    ;




/*---Produccion de estilos---*/


lista_propiedades            ::=  MENOR  COLOR_KEY  IGUAL  valor_color:color1
CIERRE_ETIQUETA

                        | MENOR BACKGROUND COLOR_KEY IGUAL valor_color:color2
CIERRE_ETIQUETA


                                  |  MENOR  FONT  FAMILY  IGUAL  TIPOGRAFIA:tipo
CIERRE_ETIQUETA

                        | MENOR TEXT SIZE IGUAL valor_numerico:num1 MAYOR

                                 | MENOR BORDER COMA valor_numerico:num2 COMA
valor_borde:grosor COMA COLOR_KEY IGUAL valor_color:colores3 CIERRE_ETIQUETA


                        ;
```

DIAGRAMA DE CLASES

Este es uno de los diagramas que describe la jerarquía principal que sigue el traductor para poder generar el código compilado. Simplemente es una jerarquía con métodos integrados para poder describir cómo es que se genera el árbol de derivación y este a su vez permite ejecutar el código en base a código Java al ser todo una misma herencia. **Cabe destacar que principalmente se pensó para poderse ejecutar en algunas acciones. Y por ello tiene muchos métodos que permiten delegar a cada clase su propio rol de validarse y de compilarse.**

Puedes verlo más a detalle en el link:

https://app.diagrams.net/#G1lN6Cfe04bDY1UQjqDtg8LLg_n23sumbg#%7B%22pageId%22%3A%229FyhZFpsurD1-RChVbB9%22%7D